

# What software does social science run on?

Measuring validated use in replication code

Gaurav Sood

## Contents

|           |                                                                             |           |
|-----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>The credit problem</b>                                                   | <b>1</b>  |
| <b>2</b>  | <b>What validated use means</b>                                             | <b>2</b>  |
| <b>3</b>  | <b>What we found</b>                                                        | <b>4</b>  |
| 3.1       | The most-used research software formats tables . . . . .                    | 4         |
| 3.2       | Stata is the majority language and the one nobody counts . . . . .          | 5         |
| 3.3       | Concentration differs by language, for reasons about denominators . . . . . | 5         |
| 3.4       | Some of the most-used software is in no registry at all . . . . .           | 5         |
| <b>4</b>  | <b>What better counts are for</b>                                           | <b>6</b>  |
| <b>5</b>  | <b>Sampling frame</b>                                                       | <b>6</b>  |
| <b>6</b>  | <b>Measurement</b>                                                          | <b>7</b>  |
| <b>7</b>  | <b>Validation</b>                                                           | <b>8</b>  |
| <b>8</b>  | <b>Limitations</b>                                                          | <b>10</b> |
| <b>9</b>  | <b>Data and code availability</b>                                           | <b>11</b> |
| <b>10</b> | <b>Reproducibility</b>                                                      | <b>12</b> |
| <b>11</b> | <b>References</b>                                                           | <b>12</b> |

## 1 The credit problem

The most widely used piece of third-party software in social science replication code is a table formatter. `estout` appears in 61% of the Stata deposits we collected, ahead of every estimator. Whatever we currently reward, it is not what we think.

We could not have told you that a year ago, because nobody could count it. That is the problem this paper is about.

Research runs on software that its authors are not credited for (Howison and Herbsleb 2011; Smith et al. 2016), and credit is what gets software written. Someone choosing between a paper and a package is choosing between something their department knows how to value and something it does not. Better counts of use would change that calculation, which makes the counts worth getting right (Sood 2021).

The counts available today are weak in a specific way. Downloads measure fetches, not use, and reward frequent releases; a package can double its numbers by cutting more versions. Prose mentions in articles (Schindler et al. 2022) capture what an author remembered to name in a methods section. Neither establishes that the software ran.

Replication code does. A deposit attached to a published paper at a journal whose data-and-code policy an editor actually verifies is the strongest routine evidence that a package was used in research (Stodden et al. 2018; Vilhuber 2020). We call this **validated use**, and this paper is about measuring it across R, Python and Stata.

## 2 What validated use means

The estimand:

Among replication deposits in collection  $J$  containing at least one analyzable script in language  $L$ , the share that statically reference package  $P$ .

The unit is the deposit, not the mention. A deposit calling `ggplot2` two hundred times is one user of `ggplot2`, and a mention-weighted count would rank whichever deposits happen to be largest.

Static reference is not execution. A deposit may load a package in a superseded script, inside a dead branch, or in setup code that never runs, and code kept out of the deposit is invisible to us. A replication package is a curated artifact rather than a forensic copy of an environment (Trisovic et al. 2022). Reachability from master scripts is reported as a sensitivity analysis, which bounds dead code without pretending to observe execution.

Reference also means direct reference. A deposit loading `fixest` also runs `Rcpp` and `sandwich`, and neither is counted. That is the right choice for a credit metric: what an author chose and what their choice pulled in are different quantities, and crediting the closure would hand the top of the table to whatever the popular packages happen to depend on.

Expanding the closure is a separate exercise, and possible in two of our three languages. R declares dependencies in `DESCRIPTION`, Python in a distribution’s `requires_dist`, and both can be walked mechanically. An SSC `.pkg` manifest records them, when it records them at all, as prose in a free-text field. The `reghdfe` manifest reads `Requires: Stata version 11.2 and ftools from SSC (q.v.)`, so extracting a graph from it means parsing English.

The closest prior work executed R code from over 2,000 Harvard Dataverse deposits to study reproducibility (Trisovic et al. 2022). We take a similar corpus and ask which libraries the code references, across three languages rather than one.

**Table 1:** Most used third-party packages, by language

| language | package      | deposits | of   | share |
|----------|--------------|----------|------|-------|
| stata    | estout       | 657      | 1075 | 61%   |
| stata    | reghdfe      | 375      | 1075 | 35%   |
| stata    | outreg2      | 263      | 1075 | 24%   |
| stata    | coefplot     | 223      | 1075 | 21%   |
| stata    | binscatter   | 127      | 1075 | 12%   |
| stata    | ivreg2       | 109      | 1075 | 10%   |
| stata    | ivreghdfe    | 99       | 1075 | 9%    |
| stata    | winsor2      | 87       | 1075 | 8%    |
| stata    | spmap        | 87       | 1075 | 8%    |
| stata    | distinct     | 81       | 1075 | 8%    |
| r        | dplyr        | 197      | 392  | 50%   |
| r        | ggplot2      | 158      | 392  | 40%   |
| r        | tidyverse    | 131      | 392  | 33%   |
| r        | haven        | 126      | 392  | 32%   |
| r        | data.table   | 106      | 392  | 27%   |
| r        | readxl       | 105      | 392  | 27%   |
| r        | tidyr        | 87       | 392  | 22%   |
| r        | stringr      | 83       | 392  | 21%   |
| r        | scales       | 68       | 392  | 17%   |
| r        | xtable       | 66       | 392  | 17%   |
| python   | pandas       | 185      | 224  | 83%   |
| python   | numpy        | 174      | 224  | 78%   |
| python   | matplotlib   | 129      | 224  | 58%   |
| python   | scipy        | 98       | 224  | 44%   |
| python   | statsmodels  | 54       | 224  | 24%   |
| python   | scikit-learn | 42       | 224  | 19%   |
| python   | seaborn      | 38       | 224  | 17%   |
| python   | numba        | 20       | 224  | 9%    |
| python   | nlTK         | 19       | 224  | 8%    |
| python   | tqdm         | 18       | 224  | 8%    |

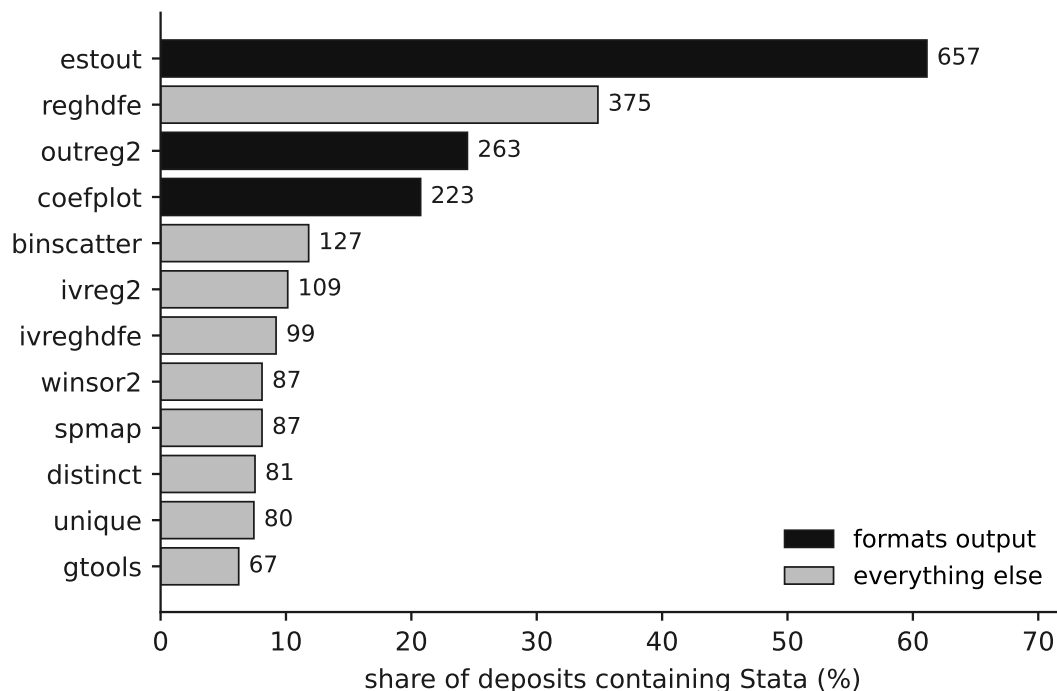
*Note:* The ten packages appearing in the most deposits in each language. *deposits* counts deposits referencing the package; *of* is the number of deposits containing analyzable code in that language, which is the only denominator these shares are comparable against. A deposit counts once however many times it calls the package.

### 3 What we found

Every count is a share of the deposits **at risk**, meaning those that contain analyzable code in that language. A share against any other denominator is not comparable to anything.

Literate documents make that denominator harder to compute than it looks. A Jupyter notebook's file type is `ipynb` and the code inside it is Python, so counting file types alone puts a deposit whose only Python lives in a notebook into the numerator of every Python package it loads and into the denominator of none. A deposit therefore counts as at risk for a language when it holds an analyzable file in that language *or* when a reference in that language came out of it. The correction is exact for R, Python and Stata: every deposit with chunks in one of them yields at least one reference in it.

#### 3.1 The most-used research software formats tables



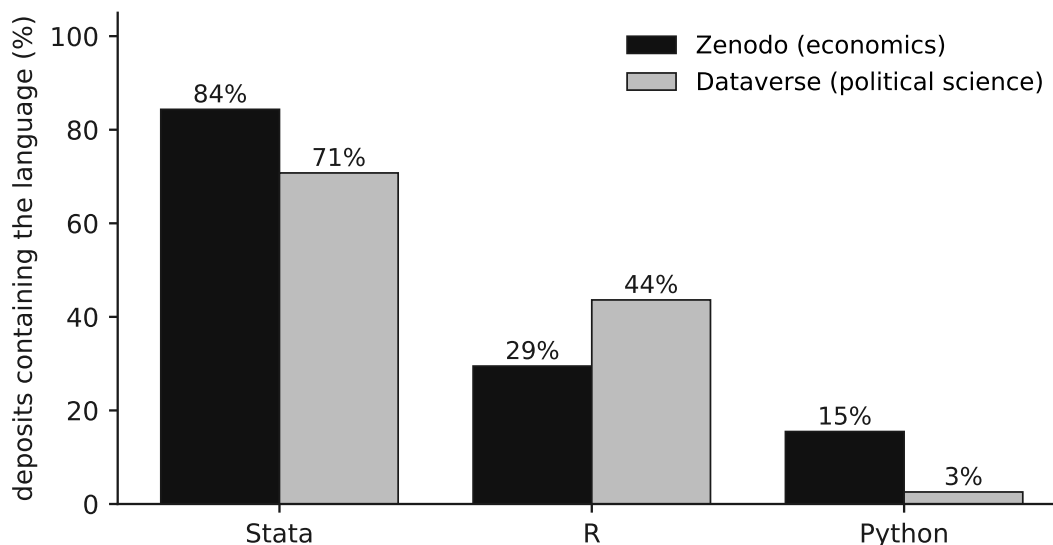
**Figure 1:** The twelve most used Stata packages, by share of deposits

*Note:* Packages whose purpose is presenting results rather than estimating anything are filled dark. Counts to the right of each bar are deposits. Shares are of deposits containing analyzable Stata.

Of the ten most widely used Stata packages (Table 1), 3 exist to format output rather than to estimate anything: `estout`, `outreg2`, `coefplot`. The ordering is the finding, and Fig. 1 is the ordering. The single most used is `estout`, in 61% of Stata deposits, ahead of every regression command.

This is worth sitting with if the point of counting is credit. A large part of what research depends on is the machinery for turning estimates into a publishable exhibit, and that machinery is written by people whose CVs say they wrote a package nobody cites.

### 3.2 Stata is the majority language and the one nobody counts



**Figure 2:** Languages by repository, and so by discipline

*Note:* Share of deposits containing at least one analyzable file in each language. Zenodo’s verified collections are economics journals; the Harvard Dataverse collections are mostly political science. A deposit may contain more than one language, so the bars within a repository do not sum to one hundred.

1,077 deposits contain analyzable Stata against 382 with R and 203 with Python, roughly 2.8 Stata deposits for every R one, and [Fig. 2](#) shows the split is disciplinary rather than incidental.

Studies of research software cover R and Python, the two languages with public registries. That omission is not a random sample of the field. It removes the language most of this literature is written in, and it removes it precisely because counting it is harder, which is a bad reason for a discipline to be invisible.

### 3.3 Concentration differs by language, for reasons about denominators

`pandas` appears in 83% of Python deposits, because Python in a replication package almost always means a dataframe pipeline. Stata’s most common package reaches 61% with a much longer tail, because Stata does natively what a Python deposit must import a library to do.

Reading those two numbers as “Python users are more consistent” would be a mistake. They are shares of different kinds of deposit, and the languages are not substitutes for one another here.

### 3.4 Some of the most-used software is in no registry at all

`grc1leg` is the most widespread Stata command we cannot resolve to any package, and it appears in 103 deposits. It is not obscure. It is distributed from StataCorp’s own website and indexed nowhere. Below it the pattern holds: `dcdensity` from its author’s page, `btscs` from a replication archive, `prvalue` and `listcoef` from a suite that ships with a textbook.

A credit system built on registries cannot see any of them, and no amount of indexing SSC more carefully will help, because they were never on SSC. This is the ceiling on what anyone can currently count about Stata, and it is a fact about the ecosystem rather than a gap in our method. R and Python have nothing comparable: a package outside CRAN or PyPI is unusual enough to be remarkable.

## 4 What better counts are for

Two uses, and the first is the one the counts were built for.

Software authors can be credited against something defensible. A package in 61% of the Stata deposits at policy-verified journals is a stronger claim on a CV than a download count, and it is a claim a committee can check. The tables here are released so that any package’s number can be looked up rather than asserted.

The second use is targeting. Work on software reliability has to choose what to examine, and choosing by reputation reproduces whatever the field already believes. Our sibling project **kasauti** runs a battery of probes against every release of a package to find where results silently changed, and it picks its targets partly by how often a function appears in real code. Validated-use counts answer that question on purpose instead of by hand, and they point at the packages where a quiet change would touch the most published work.

## 5 Sampling frame

The inclusion rule is not ours. We take the journal list maintained by the **Social Science Data Editors** in the Data and Code Availability Standard: journals whose data-and-code policy the editors *actively verify*. That is a criterion a reader can check, and it avoids the trap of a frame assembled from whatever a keyword search happened to return.

The frame spans two repositories, which is not a convenience: they hold different disciplines. Zenodo’s verified collections are economics; Harvard Dataverse’s journal collections are mostly political science, and a study of only one would describe a field while claiming to describe social science.

| repository        | collections | deposits      |
|-------------------|-------------|---------------|
| Zenodo            | 6           | 1,601         |
| Harvard Dataverse | 74          | 14,674        |
| <b>total</b>      |             | <b>16,275</b> |

The Dataverse figure is enumerated rather than declared. One request returns a journal collection’s entire dataset list, so these are deposits we counted, not an estimate we were given, and where a collection was too large for that single listing, it was paged instead.

Two things the enumeration found are worth recording, because a frame that only reports its successes is not a frame. One journal collection present in a 2024 snapshot,

`regionalstatistics`, no longer exists: Harvard answers “Can’t find dataverse with identifier”. And six collections returned nothing on one pass, of which five succeeded minutes later, transient, and distinguished from the one that is genuinely gone, because retrying the first kind works and retrying the second never will.

10 DCAS journals are recorded as **unlocated** rather than omitted: eight AEA titles live on openICPSR, and two we could not place. A coverage gap belongs in the frame as a row, not in a silence.

Every collection is verified before use. Zenodo does not reject an unknown `communities=` filter, it ignores it and returns all 7.1 million records, so an unverified slug would substitute the entire archive for one journal, an error shaped like abundance.

Of 1,601 Zenodo deposits attempted, 1,494 (93%) yielded files. [Table 2](#) itemises the rest rather than summarising them, because the reasons are not interchangeable. A size cap is a choice we made, an unreassembled multi-part archive is a format we declined to handle, and a failure is a bug or an outage.

**Table 2:** Why a deposit yielded no code

| reason                              | deposits |
|-------------------------------------|----------|
| archive over the size cap           | 66       |
| deposit contains no code files      | 39       |
| multi-part archive, not reassembled | 2        |

*Note:* Zenodo deposits that produced no analyzable file, by reason. The size cap and the unreassembled multi-part archives are choices we made and are recoverable; the single failure is a zip using a compression method Python does not implement.

## 6 Measurement

| language                    | mechanism                                                |
|-----------------------------|----------------------------------------------------------|
| R                           | <code>tree-sitter</code> syntax walk                     |
| Python                      | <code>stdlib ast</code> , <code>tokenize</code> fallback |
| Stata                       | purpose-built lexer + SSC command→package index          |
| <code>.Rmd/.qmd/.Rnw</code> | knitr chunk splitter, routed per engine                  |
| <code>.ipynb</code>         | per-cell, kernel language from metadata                  |

Two choices carry most of the weight.

**Strings and comments are excluded structurally, not by pre-stripping.** In a syntax tree, `cat("run library(dplyr)")` is a call whose argument is a string node; those bytes never form a call, so a walk over call nodes cannot see them. Regex-based detection has no such guarantee, and the difference is measurable (see the validation section).

**Stata required building an artifact that did not exist.** R has CRAN and Python has PyPI, but Stata has no machine-readable registry. We reconstruct a command→package index

from SSC distribution manifests, 3,967 packages and 7,468 commands, and release it. Without it, Stata is unmeasurable, which is why prior work has omitted the language social science uses most.

## 7 Validation

**Table 3:** Mentions that could name a package, and those we could not resolve

| language | candidate mentions | unresolved | rate  |
|----------|--------------------|------------|-------|
| stata    | 208997             | 12950      | 6.2%  |
| r        | 47187              | 633        | 1.3%  |
| python   | 15043              | 2041       | 13.6% |

*Note:* Restricted to mentions that could name a third-party package, so Stata builtins, standard libraries and programs a deposit defines itself are excluded from both columns. An unresolved mention is one we cannot attribute, not evidence that no package exists.

We report what these checks can establish and no more.

**Registry resolution** is coverage, not precision: a local module can share a name with a real distribution. 5.8% of third-party-candidate mentions resolve to no registry, and every such name is enumerated rather than dropped.

The denominator matters more than the rate. Against *all* mentions the same figure is 0.5%, which would be a flattering way to describe the identical data: it divides by 2,983,005 Stata builtin calls that were never candidates for registry resolution and could not have been unresolved. We report the rate against mentions that could name a package, which is 12 times larger and is the number a reader should use to discount our coverage.

**So does the unit.** Per deposit – the unit this paper uses everywhere else, for the reason it gives for package rankings – the distribution is not centred anywhere near the pooled figure:

| statistic               | unresolved share |
|-------------------------|------------------|
| pooled over mentions    | 5.8%             |
| median deposit          | 0.0%             |
| 90th-percentile deposit | 19.1%            |
| deposits fully resolved | 57%              |

57% of deposits have **every** candidate mention resolved. The pooled figure is not wrong, but it describes a minority of deposits and should be read as an upper bound on how much of a typical deposit we cannot name, not as a typical value.

The rate differs by language in a way a pooled figure hides, and the reasons differ with it.

**Stata carries almost all of it**, 83% of unresolved mentions, and what remains there is mostly not our error. It is a property of the Stata ecosystem, and it is the second half of the argument the command→package index makes.

The most widespread unresolved command is `grc1leg`, in 103 deposits: a real, widely used program distributed from StataCorp’s own website and indexed in no machine-readable registry, SSC included. Below it the pattern holds, `dcdensity` from its author’s page, `btscs` from a replication archive, `prvalue` and `listcoef` from the SPost suite distributed with a textbook. These are not obscure. They are ordinary tools in political science and economics, and no amount of indexing SSC will find them, because they were never on SSC.

That is worth stating plainly rather than reporting as coverage. R has CRAN, Python has PyPI, and a package outside those registries is unusual enough to be remarkable. Stata’s third-party ecosystem has a substantial tail that lives on personal pages, textbook companion sites and the vendor’s own server, and any measurement of Stata software use, ours or anyone’s, is bounded by that rather than by how carefully the indexing is done. The remainder of the unresolved names are lexer error, enumerated in the limitations below rather than netted out.

**Names that no package could ever have.** A mention is unresolved when we cannot name a package for it, which is not the same as there being one. Two constructs make that distinction explicit rather than letting it inflate the rate: 1,324 Python relative imports (`from .models import Constants` names a module in the deposit, not a distribution) and 350 names built at run time (`library(pkg, character.only = TRUE)`, where the package is a variable and no static analysis can know it). Both are reported in their own categories. Folding them into `unknown`, which an earlier version of this pipeline did, overstates the unresolved rate and, worse, credits some of them to real packages that happen to share the name: `models`, `results` and `log` are all distributions on PyPI as well as things people call their own files.

**The official-command list is measured, not curated.** Stata’s builtin list is the one place where an omission is not neutral: a command that official Stata provides but our list lacks falls through to the SSC index, and if some package happens to ship a file of that name, it is attributed there. Rather than curate from memory we checked every command in the corpus against StataCorp’s public help server, which is authoritative in the only way that matters, it is the vendor’s own answer.

This mattered more than expected. It moved 358 commands out of “resolves to nothing”, including `ds`, `pca` and `testparm`, and it removed every case where a command claimed by both official Stata and an SSC package had been credited to the package. With the curated list alone the unresolved rate on this same corpus is 10.1% rather than 5.8%, and 1,071 mentions would have been credited to an SSC package that merely ships a file of the same name. Much of what looked like a coverage gap was our own ignorance of Stata, not a limit of the method.

The check is not a status code. Every query returns HTTP 200, including for names that do not exist, so existence is read from the body text; and the server documents concepts and functions as well as commands, so pages from the User’s Guide and function manuals are excluded. Without that second filter the system variables `_n` and `_b` and the function `e()` are reported as official Stata commands.

**Cross-tool agreement** with `renv::dependencies()` (1.2.3) over 377 corpus R deposits gives a mean per-deposit Jaccard of 0.89 and a pooled Jaccard of 0.85; 53% of deposits agree exactly. Both are reported because they answer different questions: the mean weights every deposit equally, the pooled figure every package-deposit pair, so a headline cannot be flattered by deposits with one obvious package in them.

This is *convergent validity*, *not accuracy*. `renv` performs the same kind of static scan over the same files and shares the same blind spots, a package named only in a string, or loaded through a variable, is invisible to both, so agreement measures whether two implementations of one idea agree, not whether either is right. What it can catch is an implementation error on our side large enough to show against an independent codebase, which is why the per-deposit disagreements are written out rather than reduced to the headline.

**Syntactic accuracy has a true oracle, and we use it.** For “does this file contain a call to `library` whose argument is `x`”, R’s own parser is not a peer but the definition of the answer, so disagreement is our error and the error rate is exact rather than indicative. Running R’s parser over 18,120 corpus .R files and recursing its expression trees gives precision 1.0000 and recall 1.0000 across 25,969 name-file pairs.

A further 306 files R itself could not parse. Those are excluded rather than scored: an un-parseable file is not a file with no dependencies, and counting it as one would credit us for finding nothing in it.

**How that figure was reached matters as much as its value.** Perfect agreement is what a comparison looks like when the comparison is broken, so the working is worth stating. Five disagreements were adjudicated by consulting R’s behaviour directly, and four of the five were the oracle’s fault: it unboxed a one-element result into a bare string, so a lone `LearnBayes` was read as eight packages named after its letters; it skipped every `character.only = TRUE` call, though a string literal there is still a literal; it could not see through `base::library(c)`, because such a call’s head is itself a call; and it never descended into a function’s formals, hiding every `pkg::fun()` used as a default argument. One was ours: `"m"::baz()` is legal R and we dropped it. Where the two disagreed about `requireNamespace(pkg)` we ran R rather than reasoning: `library(pkg)` with `pkg <- "stats"` errors, so the symbol is the name, while `requireNamespace(pkg)` returns `TRUE`, so the symbol is a variable.

A number tuned against the same data it scores is worth little, so the extractor is also checked against ground truth we did not write: `renv`’s own test fixture, which enumerates thirteen loader spellings and states in its comments which must be found. That check failed where the oracle had not, because a blind spot two implementations share is indistinguishable from agreement, it found `xfun::pkg_attach(c("g", "h"))` and `pkg_attach2("i", "j")` unhandled. Both now are, and the fixture is a test.

This bounds one thing exactly, whether our tree-sitter walk sees what R sees, and nothing else. It says nothing about Stata, nothing about resolution against CRAN, and nothing about whether a `library()` call means the analysis used the package.

**What we do not claim.** We have not hand-labelled a probability sample, so we report no human-validated precision or recall against research *use*. That is a deliberate deferral; the labelling frame is designed and can be run.

## 8 Limitations

Static reference is not execution. The reachability analysis bounds that gap rather than closing it.

Deposits whose only content exceeds the archive size cap yield no code, and the tail this excludes is not data without code. Code files scale with archive size: a median of 13 code files in the smallest quartile of archives against 46 in the largest. The gap therefore runs against large, code-heavy projects rather than against large datasets.

Multi-part archives are not reassembled. 2 deposits split a package across `.z01`, `.7z.002` or `.partN.rar` segments. Only the tail segment carries the central directory, so it downloads at exactly the size the repository states, lists every member, and fails on the first read. That looks like a corrupt download and is not one. We count these rather than retrying them, since no number of retries can assemble one segment.

Stata resolution reflects today's status, not the status when the code was written. A command that was user-written in 2010 and later absorbed into official Stata resolves against what it is now, which can manufacture a trend for anyone reading these counts over time.

Two false positives survive, and we name them rather than suppress them. SSC packages whose only command is a single common token attract stray matches from Mata-style bare assignments outside a `mata:` block, which the lexer cannot recognise as Mata. Currently `fastcd` (command `c`) in 2 deposits. A rule against single-letter commands would hide the next instance instead of surfacing it, and because the count is computed, the list shrinks on its own as causes are fixed.

Three lexer gaps remain open and countable. Stata keywords reach command position in contexts the lexer does not model: `in` and `using` after an old-style `/* */` line continuation, and `str` inside an `infile dictionary` block, where a line such as `str FACT_ID 5-12` declares a storage type rather than calling anything. Together that is 1,030 mentions across 18 deposits. They inflate the unresolved rate and nothing else, because a keyword cannot collide with an SSC package name and so none is misattributed. Reporting the number is worth more than a blocklist that would suppress the symptom and leave the contexts unmodelled.

MATLAB, SAS and SPSS are reported as language presence only. MATLAB has no import statement and no resolvable registry, so attributing toolboxes would be unbounded error.

## 9 Data and code availability

The Stata command-to-package index is deposited at [10.5281/zenodo.21926100](https://zenodo.org/record/2192610) under CC0, with 7,468 commands across 3,967 packages. It is released separately from this paper because its usefulness does not depend on this corpus: anyone counting Stata needs the same mapping.

The extraction pipeline, the resolver and the frame are at <https://github.com/recite/softverse>. The per-package counts are published as CSV under CC0 at <https://recite.github.io/softverse/data/>, and every one of the 1,806 packages can be looked up by name at <https://recite.github.io/softverse/lookup/> rather than read off the top ten in Table 1. That is the point of producing the counts: a credit metric nobody can query is an assertion.

## 10 Reproducibility

The corpus, the registry snapshots and the extractor version are pinned. Every mention carries the extractor version, grammar version and registry lock that produced it, plus its source line and a snippet, so any number here can be traced to the text it came from without re-running anything.

**Table 4:** Pinned registry snapshots

| registry      | sha256           |
|---------------|------------------|
| cran          | aefeb12f742cad24 |
| cran_archive  | 5e267904e40ed060 |
| bioconductor  | 3ffed30246d706ae |
| pypi          | 751e02220c921dea |
| julia_general | 9cf5a9b5f71da392 |

*Note:* Every mention carries the lock identifier of the registry snapshot it was resolved against, so a count can be reproduced against the registry as it stood rather than as it stands.

## 11 References

- Howison, James, and James D. Herbsleb. 2011. “Scientific Software Production.” *Proceedings of the ACM 2011 Conference on Computer Supported Cooperative Work*, ahead of print. <https://doi.org/10.1145/1958824.1958904>.
- Schindler, David, Felix Bensmann, Stefan Dietze, and Frank Krüger. 2022. “The Role of Software in Science: A Knowledge Graph-Based Analysis of Software Mentions in PubMed Central.” *PeerJ Computer Science* 8: e835. <https://doi.org/10.7717/peerj-cs.835>.
- Smith, Arfon M., Daniel S. Katz, and Kyle E. Niemeyer. 2016. “Software Citation Principles.” *PeerJ Computer Science* 2: e86. <https://doi.org/10.7717/peerj-cs.86>.
- Sood, Gaurav. 2021. *Country: Incentivizing More and Better Software*. <https://www.gojiberies.io/country-incentivizing-more-and-better-software/>.
- Stodden, Victoria, Jennifer Seiler, and Zhaokun Ma. 2018. “An Empirical Analysis of Journal Policy Effectiveness for Computational Reproducibility.” *Proceedings of the National Academy of Sciences* 115 (11): 2584–89. <https://doi.org/10.1073/pnas.1708290115>.
- Trisovic, Ana, Matthew K. Lau, Thomas Pasquier, and Mercè Crosas. 2022. “A Large-Scale Study on Research Code Quality and Execution.” *Scientific Data* 9 (1): 60. <https://doi.org/10.1038/s41597-022-01143-6>.
- Vilhuber, Lars. 2020. “Reproducibility and Replicability in Economics.” *Harvard Data Science Review* 2 (4). <https://doi.org/10.1162/99608f92.4f6b9e67>.